

Frontend Architecture

Overview

The HRMS frontend is built with vanilla JavaScript using a modern, component-based architecture that emphasizes performance, maintainability, and user experience without framework dependencies.

Architecture Philosophy

Framework-Free Approach

- **No Dependencies:** Pure vanilla JavaScript, HTML5, and CSS3
- **Lightweight:** Fast loading and minimal bundle size
- **Future-Proof:** No framework lock-in or version compatibility issues
- **Performance:** Direct DOM manipulation without abstraction overhead

Modern JavaScript Patterns

- **ES6+ Features:** Classes, modules, async/await, destructuring
- **Component System:** Reusable UI components with encapsulation
- **Service Layer:** Centralized API communication
- **Event-Driven:** Pub/sub pattern for component communication

Project Structure

```
frontend/
├── css/                                # Stylesheets
│   ├── variables.css                  # CSS custom properties
│   ├── base.css                      # Reset and base styles
│   ├── layout.css                    # Layout and grid systems
│   ├── components.css                # UI component styles
│   ├── utilities.css                 # Utility classes
│   ├── animations.css                # Animation and transitions
│   └── landing.css                   # Landing page specific styles
├── js/                                # JavaScript modules
│   ├── api.js                        # API service layer
│   ├── auth.js                      # Authentication handling
│   ├── components/                   # Reusable UI components
│   │   ├── header.js                # Header component
│   │   ├── sidebar.js               # Navigation sidebar
│   │   ├── toast.js                 # Notification system
│   │   └── loader.js                # Loading indicators
│   ├── pages/                        # Page-specific logic
│   │   ├── dashboard.js             # Dashboard functionality
│   │   ├── employees.js             # Employee management
│   │   ├── attendance.js            # Attendance tracking
│   │   ├── leave.js                 # Leave management
│   │   └── payroll.js               # Payroll processing
│   └── utils/                        # Utility functions
│       ├── animations.js             # Animation helpers
│       ├── validation.js             # Form validation
│       └── helpers.js                # General utilities
├── *.html                             # Application pages
└── assets/                             # Static assets
```

Core Architecture Components

1. API Service Layer

```
// api.js - Centralized API communication
class ApiError extends Error {
  constructor(code, message, details = null) {
    super(message);
    this.code = code;
    this.details = details;
  }
}

const api = {
  async request(endpoint, options = {}) {
    const url = `${API_BASE}${endpoint}`;

    const config = {
      headers: {
        'Content-Type': 'application/json',
        ...options.headers
      },
      credentials: 'include', // Include session cookies
      ...options
    };

    try {
      const response = await fetch(url, config);
      const data = await response.json();

      if (!data.success) {
        throw new ApiError(
          data.error?.code || 'UNKNOWN_ERROR',
          data.error?.message || 'An error occurred',
          data.error?.details
        );
      }

      return data;
    } catch (error) {
      if (error instanceof ApiError) {
        throw error;
      }

      throw new ApiError('NETWORK_ERROR', 'Network error. Please check your connection.');
```

Features:

- Centralized error handling
- Automatic JSON parsing
- Session cookie management
- Request/response interceptors

- Type-safe error objects

2. Component System

```
// components/toast.js - Notification system
class Toast {
  constructor() {
    this.container = this.createContainer();
    this.toasts = new Map();
  }

  createContainer() {
    let container = document.getElementById('toast-container');
    if (!container) {
      container = document.createElement('div');
      container.id = 'toast-container';
      container.className = 'toast-container';
      document.body.appendChild(container);
    }
    return container;
  }

  show(message, type = 'info', duration = 5000) {
    const id = Date.now() + Math.random();
    const toast = this.createToast(id, message, type);

    this.container.appendChild(toast);
    this.toasts.set(id, toast);

    // Auto-remove after duration
    setTimeout(() => this.remove(id), duration);

    return id;
  }

  createToast(id, message, type) {
    const toast = document.createElement('div');
    toast.className = `toast toast-${type}`;
    toast.dataset.id = id;

    toast.innerHTML = `
    <div class="toast-content">
      <span class="toast-icon">${this.getIcon(type)}</span>
      <span class="toast-message">${message}</span>
      <button class="toast-close" onclick="window.toast.remove(${id})">×</button>
    </div>
  `;

    return toast;
  }

  remove(id) {
    const toast = this.toasts.get(id);
    if (toast) {
      toast.classList.add('removing');
      setTimeout(() => {
        if (toast.parentNode) {
          toast.parentNode.removeChild(toast);
        }
        this.toasts.delete(id);
      }, 300);
    }
  }

  getIcon(type) {
    const icons = {
      success: '✓',
      error: '✗',
      warning: '⚠',
    };
  }
}
```

```

        info: 'i'
      };
      return icons[type] || icons.info;
    }
  }

// Global toast instance
window.toast = new Toast();

```

Component Features:

- Encapsulated functionality
- Reusable across pages
- Event-driven communication
- Lifecycle management
- CSS animation integration

3. Page Controllers

```

// pages/employees.js - Employee management page
class EmployeePage {
  constructor() {
    this.employees = [];
    this.filters = {
      status: 'all',
      department: 'all',
      search: ''
    };

    this.init();
  }

  async init() {
    this.bindEvents();
    this.setupFilters();
    await this.loadEmployees();
    this.renderEmployees();
  }

  bindEvents() {
    // Add employee button
    document.getElementById('add-employee-btn')?.addEventListener('click', () => {
      this.showAddEmployeeModal();
    });

    // Search input
    document.getElementById('employee-search')?.addEventListener('input', (e) => {
      this.filters.search = e.target.value;
      this.filterAndRender();
    });

    // Status filter
    document.getElementById('status-filter')?.addEventListener('change', (e) => {
      this.filters.status = e.target.value;
      this.filterAndRender();
    });

    // Department filter
    document.getElementById('department-filter')?.addEventListener('change', (e) => {
      this.filters.department = e.target.value;
      this.filterAndRender();
    });
  }

  async loadEmployees() {
    try {
      const response = await api.employees.list();
      this.employees = response.data;
    } catch (error) {

```

```

    toast.show(`Failed to load employees: ${error.message}`, 'error');
  }
}

filterAndRender() {
  const filtered = this.employees.filter(employee => {
    // Status filter
    if (this.filters.status !== 'all' && employee.status !== this.filters.status) {
      return false;
    }

    // Department filter
    if (this.filters.department !== 'all' && employee.department !== this.filters.department) {
      return false;
    }

    // Search filter
    if (this.filters.search) {
      const search = this.filters.search.toLowerCase();
      const fullName = `${employee.first_name} ${employee.last_name}`.toLowerCase();
      const email = employee.email.toLowerCase();

      if (!fullName.includes(search) && !email.includes(search) &&
        !employee.employee_code.toLowerCase().includes(search)) {
        return false;
      }
    }

    return true;
  });

  this.renderEmployees(filtered);
}

renderEmployees(employeeList = this.employees) {
  const container = document.getElementById('employees-table-body');
  if (!container) return;

  if (employeeList.length === 0) {
    container.innerHTML = `
      <tr>
        <td colspan="7" class="text-center py-8">
          <div class="empty-state">
            <div class="empty-state-icon">🚫</div>
            <h3>No employees found</h3>
            <p>No employees match your current filters.</p>
          </div>
        </td>
      </tr>
    `;
    return;
  }

  container.innerHTML = employeeList.map(employee => `
    <tr class="employee-row" data-id="${employee.id}">
      <td>
        <div class="employee-info">
          <div class="employee-avatar">
            ${employee.first_name[0]}${employee.last_name[0]}
          </div>
          <div>
            <div class="employee-name">${employee.first_name} ${employee.last_name}</div>
            <div class="employee-code">${employee.employee_code}</div>
          </div>
        </div>
      </td>
      <td>${employee.email}</td>
      <td>${employee.department || '-'}</td>
      <td>${employee.designation || '-'}</td>
  `);
}

```

```

        <td>
          <span class="badge badge-${employee.status}">
            ${employee.status.charAt(0).toUpperCase() + employee.status.slice(1)}
          </span>
        </td>
        <td>${this.formatDate(employee.hire_date)}</td>
        <td>
          <div class="action-buttons">
            <button class="btn btn-sm btn-outline" onclick="employeePage.viewEmployee(${employee.id})">
              View
            </button>
            <button class="btn btn-sm btn-outline" onclick="employeePage.editEmployee(${employee.id})">
              Edit
            </button>
            <button class="btn btn-sm btn-danger" onclick="employeePage.deleteEmployee(${employee.id})">
              Delete
            </button>
          </div>
        </td>
      </tr>
    `).join('');

    // Add row animations
    this.animateRows();
  }

  animateRows() {
    const rows = document.querySelectorAll('.employee-row');
    rows.forEach((row, index) => {
      row.style.animationDelay = `${index * 0.02}s`;
    });
  }

  async deleteEmployee(id) {
    if (!confirm('Are you sure you want to delete this employee?')) {
      return;
    }

    try {
      await api.employees.delete(id);
      toast.show('Employee deleted successfully', 'success');
      await this.loadEmployees();
      this.filterAndRender();
    } catch (error) {
      toast.show(`Failed to delete employee: ${error.message}`, 'error');
    }
  }

  formatDate(dateString) {
    return new Date(dateString).toLocaleDateString();
  }
}

// Initialize page when DOM is loaded
document.addEventListener('DOMContentLoaded', () => {
  window.employeePage = new EmployeePage();
});

```

Page Controller Features:

- State management
- Event handling
- API integration
- DOM manipulation
- Error handling
- Animation coordination

4. Authentication System

```
// auth.js - Authentication management
class AuthManager {
  constructor() {
    this.user = null;
    this.permissions = [];
    this.isAuthenticated = false;
  }

  async checkAuth() {
    try {
      const response = await api.auth.me();
      this.user = response.data.user;
      this.permissions = response.data.permissions;
      this.isAuthenticated = true;
      return true;
    } catch (error) {
      this.clearAuth();
      return false;
    }
  }

  async login(email, password) {
    try {
      const response = await api.auth.login(email, password);
      this.user = response.data.user;
      this.permissions = response.data.permissions;
      this.isAuthenticated = true;

      // Redirect to dashboard
      window.location.href = '/frontend/index.html';

      return response.data;
    } catch (error) {
      throw error;
    }
  }

  async logout() {
    try {
      await api.auth.logout();
    } catch (error) {
      console.error('Logout error:', error);
    } finally {
      this.clearAuth();
      window.location.href = '/frontend/login.html';
    }
  }

  clearAuth() {
    this.user = null;
    this.permissions = [];
    this.isAuthenticated = false;
  }

  hasPermission(permission) {
    return this.permissions.includes(permission);
  }

  requireAuth() {
    if (!this.isAuthenticated) {
      window.location.href = '/frontend/login.html';
      return false;
    }
    return true;
  }

  requirePermission(permission) {
    if (!this.hasPermission(permission)) {
      toast.show('You do not have permission to perform this action', 'error');
    }
  }
}
```

```
        return false;
    }
    return true;
}
}

// Global auth instance
window.auth = new AuthManager();

// Auto-check authentication on protected pages
document.addEventListener('DOMContentLoaded', async () => {
    // Skip auth check on public pages
    const publicPages = ['login.html', 'signup.html', 'landing.html'];
    const currentPage = window.location.pathname.split('/').pop();

    if (publicPages.includes(currentPage)) {
        return;
    }

    // Check authentication for protected pages
    const isAuthenticated = await auth.checkAuth();
    if (!isAuthenticated) {
        window.location.href = '/frontend/login.html';
    }
});
```

CSS Architecture

1. CSS Custom Properties (Variables)


```

/* variables.css - Design system tokens */
:root {
  /* Primary Colors */
  --color-primary: #1F3A5F;
  --color-primary-light: #2a4a73;
  --color-primary-dark: #152a45;
  --color-secondary: #2FB7B2;
  --color-secondary-light: #3fccc7;
  --color-secondary-dark: #259e9a;

  /* Backgrounds */
  --color-bg-page: #F5F7FA;
  --color-bg-card: #FFFFFF;
  --color-bg-hover: #f0f2f5;
  --color-border: #E1E6EF;

  /* Text */
  --color-text-heading: #1C1E21;
  --color-text-body: #6B7280;
  --color-text-muted: #9CA3AF;

  /* Status Colors */
  --color-success: #22C55E;
  --color-warning: #F59E0B;
  --color-error: #EF4444;
  --color-info: #3B82F6;

  /* Spacing */
  --spacing-xs: 4px;
  --spacing-sm: 8px;
  --spacing-md: 16px;
  --spacing-lg: 24px;
  --spacing-xl: 32px;
  --spacing-2xl: 48px;

  /* Layout */
  --sidebar-width: 260px;
  --header-height: 64px;

  /* Transitions */
  --transition-fast: 150ms ease;
  --transition-normal: 250ms ease;

  /* Shadows */
  --shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.05);
  --shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
  --shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
}

```

2. Component-Based CSS

```

/* components.css - Reusable UI components */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: var(--spacing-sm) var(--spacing-md);
  border: 1px solid transparent;
  border-radius: 6px;
  font-size: 14px;
  font-weight: 500;
  text-decoration: none;
  cursor: pointer;
  transition: all var(--transition-fast);
  position: relative;
  overflow: hidden;
}

.btn-primary {
  background: var(--color-primary);
  color: white;
  border-color: var(--color-primary);
}

.btn-primary:hover {
  background: var(--color-primary-dark);
  transform: translateY(-1px);
  box-shadow: var(--shadow-md);
}

.card {
  background: var(--color-bg-card);
  border: 1px solid var(--color-border);
  border-radius: 8px;
  padding: var(--spacing-lg);
  box-shadow: var(--shadow-sm);
  transition: all var(--transition-normal);
}

.card:hover {
  transform: translateY(-2px);
  box-shadow: var(--shadow-lg);
}

```

3. Animation System

```

/* animations.css - Smooth transitions and effects */
:root {
  --ease-out-expo: cubic-bezier(0.16, 1, 0.3, 1);
  --ease-out-back: cubic-bezier(0.34, 1.56, 0.64, 1);
  --ease-liquid: cubic-bezier(0.4, 0, 0.2, 1);
}

/* Page transitions */
.main-content {
  animation: fadeSlideUp 0.5s var(--ease-out-expo);
}

@keyframes fadeSlideUp {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Staggered animations */
.data-table tbody tr {
  animation: rowSlideIn 0.4s var(--ease-out-expo) backwards;
}

.data-table tbody tr:nth-child(1) { animation-delay: 0.02s; }
.data-table tbody tr:nth-child(2) { animation-delay: 0.04s; }
.data-table tbody tr:nth-child(3) { animation-delay: 0.06s; }

@keyframes rowSlideIn {
  from {
    opacity: 0;
    transform: translateX(-20px);
  }
  to {
    opacity: 1;
    transform: translateX(0);
  }
}

```

Performance Optimizations

1. Lazy Loading

```

// Lazy load page modules
const loadPage = async (pageName) => {
  try {
    const module = await import(`./pages/${pageName}.js`);
    return module.default;
  } catch (error) {
    console.error(`Failed to load page: ${pageName}`, error);
  }
};

```

2. Event Delegation

```
// Efficient event handling with delegation
document.addEventListener('click', (e) => {
  if (e.target.matches('.btn-delete')) {
    handleDelete(e.target.dataset.id);
  }

  if (e.target.matches('.btn-edit')) {
    handleEdit(e.target.dataset.id);
  }
});
```

3. Virtual Scrolling (for large datasets)

```
class VirtualScroller {
  constructor(container, itemHeight, renderItem) {
    this.container = container;
    this.itemHeight = itemHeight;
    this.renderItem = renderItem;
    this.visibleItems = Math.ceil(container.clientHeight / itemHeight) + 2;

    this.setupScrolling();
  }

  render(data) {
    const scrollTop = this.container.scrollTop;
    const startIndex = Math.floor(scrollTop / this.itemHeight);
    const endIndex = Math.min(startIndex + this.visibleItems, data.length);

    const visibleData = data.slice(startIndex, endIndex);

    this.container.innerHTML = visibleData
      .map((item, index) => this.renderItem(item, startIndex + index))
      .join('');
  }
}
```

Build and Deployment

Development Workflow

1. **No Build Step:** Direct file serving for development
2. **Live Reload:** Browser sync for development
3. **CSS Preprocessing:** Optional PostCSS for production
4. **Asset Optimization:** Minification and compression

Production Optimization

```
// Service worker for caching
self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open('hrms-v1').then((cache) => {
      return cache.addAll([
        '/frontend/',
        '/frontend/css/variables.css',
        '/frontend/css/components.css',
        '/frontend/js/api.js',
        '/frontend/js/auth.js'
      ]);
    })
  );
});
```

This frontend architecture provides a scalable, maintainable foundation with excellent performance characteristics and modern development practices without framework dependencies.